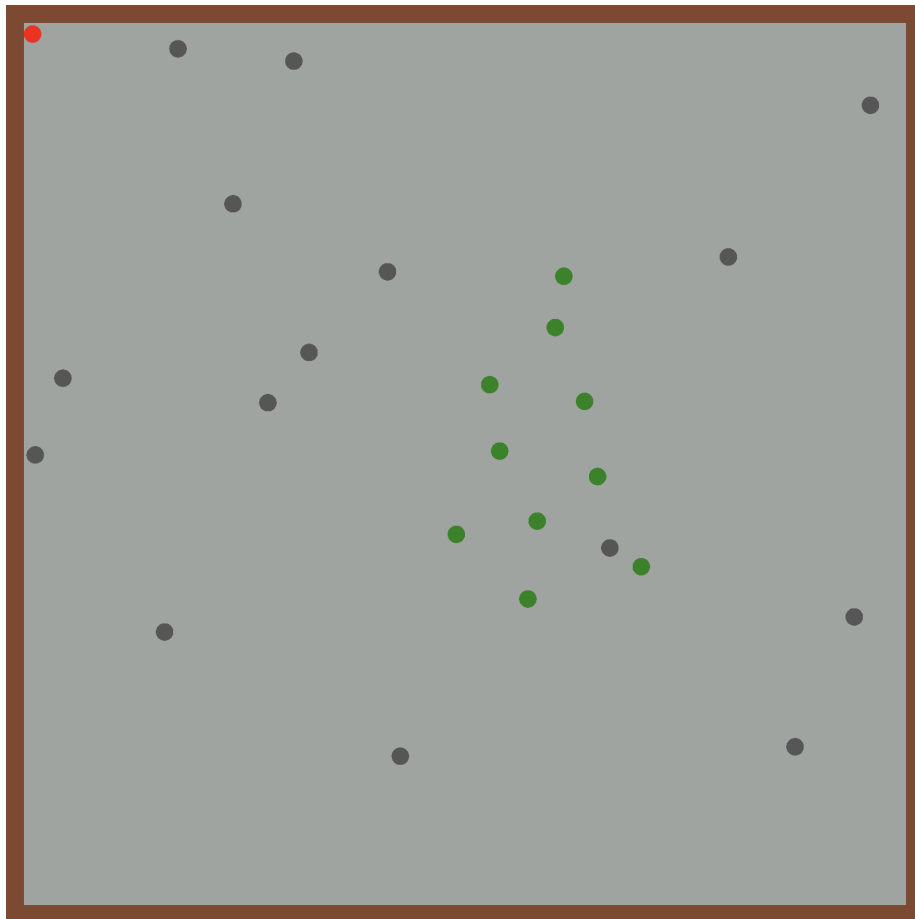


AI For Videogames Flocking Simulation

Giovanni Bollati

February 2025



Contents

1	Introduction	3
1.1	Requirements	3
2	Movement Behaviours	4
2.1	Flocking Behaviours	4
2.2	Collision Avoidance	4
2.3	Combination	5
3	Implementation	6
3.1	Boids	7
3.2	Obstacles	7
4	Application	8
5	Conclusion	9

1 Introduction

This project consists in the simulation of a flock of generic agents, called boids, that seek a target placed in a square room, while avoiding horizontally moving objects.

The implemented flocking behaviour takes inspiration from the treatment[3] made by Craig Reynolds in 1987; it consists in the blending of three different and independent behaviours: separation, cohesion and alignment.

1.1 Requirements

The environment is a simple square room; it is 100 units in width and height. The sides of the room are blocked by solid walls that limit the movement of both the boids and the obstacles. The boids and the obstacles are represented by a circle of uniform colour; for this reason, they cannot rotate, since the effects of rotation would not be visible. The boids and the obstacles are required to move at a constant velocity: the boids' speed is 10 units per second, while the obstacles' one is a random value between 5 and 20 units per second. The obstacles are placed in a random position but they must not overlap during their travel; when they collide with the walls they simply revert their direction. The target is represented by a red circle; when a boid reaches it, it changes position: it is randomly placed at a random distance of maximum 10 units from one of the four corners (excluding the current). The movement algorithms must be kinematic; in addition to the flocking behaviour, the boids must seek the target and avoid the moving obstacles.

2 Movement Behaviours

Since the speed must be constant, the behaviours are kinematic and they return a direction, which, multiplied by the constant speed, becomes the velocity. The kinematic data is composed of the current position and velocity.

The basic behaviours are seek and flee, which are used by most of the other behaviours, which are delegated behaviours. The flocking behaviours and obstacle avoidance behaviour requires kinematic data of both the agent and its neighbourhood, therefore each boid is able to collect data about its neighbourhood, which consists of other boids and obstacles.

2.1 Flocking Behaviours

The flocking behaviour consists in the weighted blending of separation, cohesion and alignment. Separation delegates to the weighted blending of a flee behaviour for each neighbour of the agent, weighted by the inverse of the distance between the agent and the neighbour. Cohesion delegates to seek towards the average position of the neighbours. Alignment neglects rotation matching since the boids are not allowed to rotate, therefore the behaviour consists in the matching of the average velocity direction of the neighbours. While the range of detection for the obstacles is fixed, the detection radius for nearby boids is calculated at runtime and it depends on the size of the flock.

2.2 Collision Avoidance

Collision avoidance consists of wall and obstacle avoidance.

Wall avoidance detects when a boid is going to collide with a wall along the component of its velocity with lower magnitude, so that it can travel towards a wall to reach the target, but it does not scratch the wall when it runs along it. Wall avoidance is simply implemented with raycasting: it returns the collision normal if a wall is detected.

The obstacle avoidance behaviour detects the nearby obstacles and predicts eventual collisions: if a collision is predicted, then the behaviour delegates to flee parameterized by the kinematics of the agent and the obstacle at the time of their closest approach; the closest approach is computed as the time needed for the boid to cover the distance to the obstacle with the velocity relative to the obstacle and projected onto the distance vector; the obstacle taken into consideration among the neighbours is the one with lower time for the closest approach. This method takes inspiration from the one described by Ian Millington in *AI for Games*[1]. The main parameters of this method are the obstacle detection radius and the collision prediction threshold, which is the distance value below which a boid and an obstacle are considered to be colliding. The main issue with the usage of this method in this context is that the boids cannot accelerate, thus they cannot avoid faster obstacles by overtaking them. Custom code is used to cover the occurrence and make the boids avoid the faster obstacles without trying to overtake them. In addition, the collision prediction

threshold must be higher than the sum of the obstacle and boid radii so that the boid reacts in advance and makes up for its constant speed. Faster obstacles can trap boids because the boids will try to escape by moving away from them, along the same direction of the obstacles' velocity, and being slower they will never escape; this occurrence is resolved by custom code that detects the trap and makes the boid escape by moving along a direction perpendicular to the obstacle velocity.

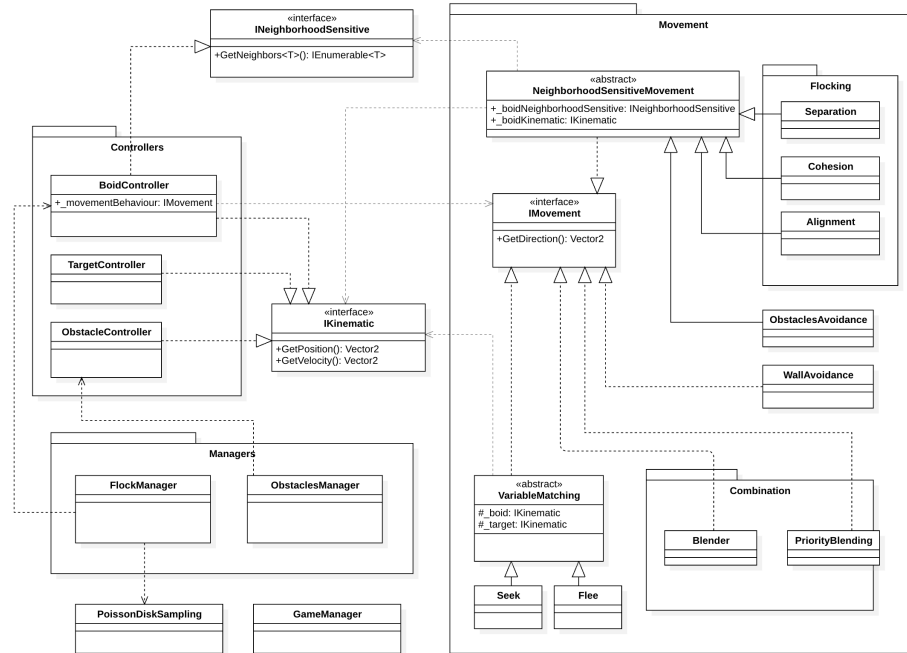
2.3 Combination

The movement behaviours are combined by both weighted blending and priority method. Separation, cohesion, alignment, seek and wall avoidance are blended together; this blending is combined with the obstacle avoidance behaviour through the priority method, with the obstacle avoidance behaviour having higher priority.

3 Implementation

The simulation is implemented with *Unity*[2] and *C#*.

The *IMovement* interface represents the ability to return a direction through the *GetDirection* method; it is implemented by every movement behaviour, including the combination methods. The *IKinematic* interface represents the ability to provide kinematic data to a movement behaviour through the *GetPosition* and *GetVelocity* methods; it is implemented by the moving entities, which are the boids and the obstacles. The movement behaviours are structured in a hierarchy. *Seek* and *Flee* are subclasses of *VariableMatching*, which holds the reference to the kinematics of the agent and its target. *ObstacleAvoidanceBehaviour* and the flocking behaviours are subclasses of *NeighborhoodSensitiveMovement*, a movement behaviour that depends on the neighbours of the agent. The *INeighbourhoodSensitive* interface represents the ability to provide data about the neighbourhood through the *GetNeighbors* method; *GetNeighbors* is a generic method and is parameterized by the caller by specifying the type of neighbours it is interested in. The boids and the obstacles are managed by the *Controller* classes, which implement the necessary interfaces and subclass *MonoBehaviour*. Their placement and generation are delegated to the *Manager* classes, which refer to boids and obstacles as prefabs. Most of the settings are held by the *Settings* scriptable object; *MonoBehaviours* that require those settings hold a reference to it. Finally, *GameManager* is a singleton that provides settings that do not need to be serialized.



3.1 Boids

The boids are GameObjects that possess a BoidController component; BoidController extends MonoBehaviour and implements IKinematic and INeighborhoodSensitive. BoidController holds a reference to IMovement, which can be any combination of movement behaviours; BoidController constructs the movement behaviours and passes them the required parameters. GetDirection is called every frame to overwrite the target direction; the target direction and the current direction are distinguished in order to smooth out the movement over multiple frames with linear interpolation. The boids detect their neighbours through the physics query *Physics2D.OverlapCircle*. The boids are spawned by the FlockManager. FlockManager spawns the boids through the Poisson Disk Sampling method[4]: the position of FlockManager defines the centre of the distribution, and a predetermined number of boids are distributed around the centre at a minimum distance between each other. This method ensures that the boids are placed in a natural manner. When FlockManager instantiates the boids, it overrides some properties, such as the target to seek; these properties are defined for each boid so that their functionality is independent from being part of a flock. FlockManager also sets the boids detection radius property, which depends on the size of the flock: the radius is equal to $\min(7, 2 * \sqrt{flock_size})$.

3.2 Obstacles

Obstacles are GameObjects that possess an ObstacleController component; ObstacleController implements IKinematic. The Obstacles are spawned by the ObstacleManager: for each obstacle, it generates a random speed between 5 and 20 units per second; then it randomly places the obstacles on the map, preventing them from overlapping during their travel.

4 Application

In the settings menu it is possible to modify the following parameters:

- number of boids: the number of boids that are part of the flock.
- boid speed: the constant speed at which each boid moves.
- boids smoothing multiplier: a value that affects how the direction changes are smoothed over multiple frames; the lower the value, the higher the smoothing.
- obstacle detection radius: the distance within which the obstacles are detected by each boid.
- collision prediction threshold: a collision is predicted when the distance between the boid and the obstacle at the time of the closest approach is less than this value.
- obstacles count: the number of obstacles that must be generated; it must be between 10 and 30.
- minimum/maximum obstacles speed: the values between which the speed is randomly set, independently for each obstacle.

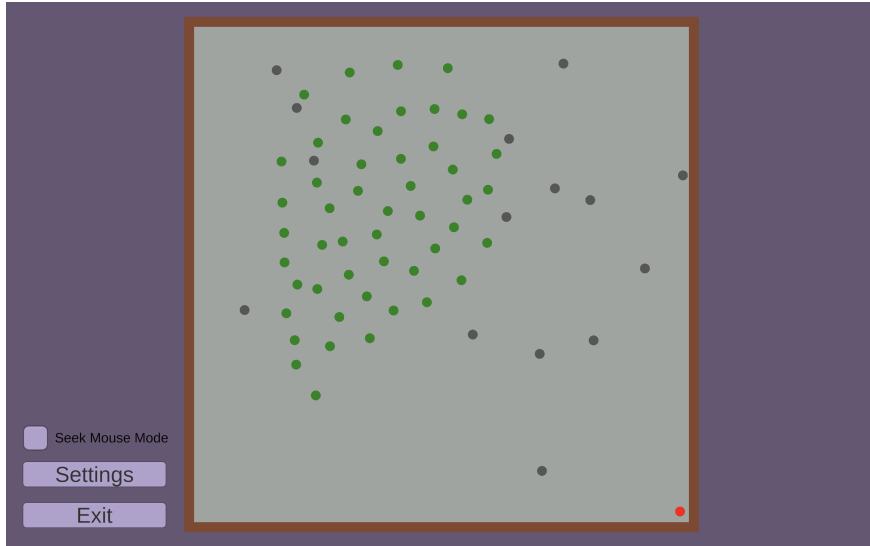
During the simulation it is possible to enable the seek mouse mode, during which the target will follow the mouse position; it is possible to exit the simulation or go back to the settings menu to change them.



The image shows a settings menu for a 'Flocking Simulation'. The title 'Flocking Simulation' is at the top center. Below it, there are several input fields and a slider arranged in two columns. The left column contains: 'number of boids' (50), 'boids smoothing multiplier' (10), 'collision prediction threshold' (4), 'number of obstacles' (10), 'obstacles min speed' (5), and 'obstacles max speed' (20). The right column contains: 'boid speed' (10), 'obstacles detection radius' (10), and a slider for 'number of obstacles' ranging from 10 to 30. At the bottom, there are two buttons: 'Start' and 'Exit'.

Parameter	Value
number of boids	50
boid speed	10
boids smoothing multiplier	10
obstacles detection radius	10
collision prediction threshold	4
number of obstacles	10
obstacles min speed	5
obstacles max speed	20

Start Exit



5 Conclusion

The movement of the boids as components of a flock is believable and works as expected; movement smoothing helps making the movement feel natural even if the speed is constant. Collision avoidance works but avoidance is not guaranteed: collisions can occur; yet, the flock manages to stay cohesive and to navigate towards the target; even if the boids collide with the obstacles, they do not get trapped and manage to continue their travel. The efficiency of the collision avoidance behaviour is dependent on the size of the flock and the number of obstacles: the bigger the flock is, the smaller the room for manoeuvre. Collision prediction does not take into account that obstacles change direction when they hit a wall; in addition, when the boids are near the wall, the wall avoidance behaviour comes into play, adding another constraint to the movement. The boids' awareness is omnidirectional: they do not have a facing direction, and they perceive obstacles and other boids in all directions; this increased awareness improves collision avoidance (e.g. a boid can avoid an obstacle that is coming from behind) while realism decreases, as long as we associate the boids to natural living beings and not to, for example, robots with omnidirectional vision.

References

- [1] Ian Millington and John Funge. *Artificial Intelligence for Games, Second Edition*. 2nd. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2009. ISBN: 0123747317.
- [2] *Unity*. URL: unity.com.
- [3] Craig W Reynolds. *Flocks, herds and schools: A distributed behavioral model*. Vol. 21. 4. 1987.
- [4] Tong Wang. “Poisson-Disk Sampling: Theory and Applications”. In: *Encyclopedia of Computer Graphics and Games*. Ed. by Newton Lee. Cham: Springer International Publishing, 2020, pp. 1–8. ISBN: 978-3-319-08234-9. DOI: [10.1007/978-3-319-08234-9_398-1](https://doi.org/10.1007/978-3-319-08234-9_398-1). URL: https://doi.org/10.1007/978-3-319-08234-9_398-1.